ELEC-H-310
Th03: Combinatorial logic circuits synthesis, hazards & introduction to sequential logic

Dragomir Milojevic
Université libre de Bruxelles

v1.3

Today

1. Combinatorial logic synthesis from verbal specifications
2. Synthesis of adder circuits
3. Circuits with multiple outputs
4. Hazards
5. Introduction to sequential systems

1. Combinatorial logic synthesis from verbal specifications

Problem of logic circuit synthesis

- Logic circuit synthesis is the process in which we transform **informal system specification** (typically verbal) into a formal representation of the system
- Formal means that the representation is: complete, precise & without any ambiguities (not really the case with verbal specs)
  - Boolean expressions (whatever the form), truth tables, K-Map, schematic etc., all these are formal representations of the system
- Verbal specification (description) typically describe the system functionality at higher abstract levels, i.e., **what** the system should do, and not **how** it should be done
  - Example: synthesise a 4-bit adder circuit
  - This is clearly a high-level description, we first need to decide how should we *effectively* compute the addition; in software we speak of algorithms, in hardware we speak of architecture
  - Once the architecture choice is made we can implement the system

Logic circuit synthesis process – formalisation

- What you have learned so far is enough to perform the logic circuit synthesis of a specific class of logic systems – combinatorial logic circuits
- This is the simplest possible model of computation, that can result in useful system, but with limited functionality as we will see
- We typically start by establishing the truth table: for a given number of inputs and all input options, we specify the output
- Truth table establishment is the process of problem formalisation and it is **absolutely essential** that this step is done correctly $\rightarrow$ system with erroneous specification will never operate correctly
- Once the truth table is established, you can derive corresponding logic expression(s) using:
  - ▹ Pen & paper – error prone, this is what we do here, or,
  - ▹ Computers – very efficient & with minimal risk of error, providing the input is correct!

Logic circuit synthesis process – circuit

- Logic expressions can be then implemented as a logic circuit
- Number of inputs fix the size of each logic function
- Number of outputs fix the number of different logic functions that need to be synthesised (N° of truth tables, K-Maps, etc.)
- Reminder:
  - Minterm is the combination of input variables for which $F = 1$ (or maxterm when $F = 0$)
  - Logic function is expressed as Sum of Minterms or Product of Maxterms (at this stage no real preference)
  - Optimisation process will result in expressions that are functionally equivalent to Sum of Minterms, but require much less logic resources; we speak of generic Sum of Products (SoP)
- If there are multiple outputs, these are handled with independent logic circuits that operate concurrently, i.e. they will react to any input change at the same time, simultaneously

Combinatorial logic circuits – formal definition

- Output O is a Boolean (logic) function f of inputs I_i **only**

$$O = f(I)$$

  where $i \in \{0, 1, \ldots, n-1\}$; i.e., there are n inputs
- Same input combination will always produce the same output
- The system **doesn't have any memory**
- There is a data flow from inputs to the outputs (signals propagate from inputs to outputs through wires and logic gates), outputs are valid as long as the **inputs are valid**
  - ▹ All inputs must be present and set to either '0' or '1', we need hard switches that make and maintain the contact; push-button switches are excluded because they do not maintain the value at the input
- This is why combinatorial logic circuits are considered the simplest possible model of computation

2. Synthesis of adder circuits

Synthesis of half- & full-adder circuits

- We have seen that the addition of the binary numbers can be implemented using elementary half- & full-adder circuits
- Half-adder takes two bits of inputs to produce a sum; since the result of the addition can be at most $(2)_{10}$ we need two bits of result: one for the sum, the other for the carry
- Half-adder takes two bits of inputs to produce a sum; since the result of the addition can be at most $(3)_{10}$ we can still represent the result with two bits: one for the sum, the other for the carry again
- We have seen the truth tables of these circuits, now we can derive the logic equations, even optimised ones to implement the circuit
- Let's use K-Maps to derive optimised logic functions of the above two circuits

Synthesis of half-adder circuit

- K-Maps for s_0 (sum bi)t, and s_1 (carry bit): left & right K-Maps below from thetruth table on the right
- All Prime Implicants are essential
- Note the use of XOR for more simplified expression

a	b	s_1	s_0
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

a_i \ b_i	0	1
0	0	1
1	1	0

$$s_0 = a_i' b_i + a_i b_i' = a_i \oplus b_i$$

a_i \ b_i	0	1
0	0	0
1	0	1

$$s_1 = a_i b_i$$

Synthesis of full-adder circuit

We do the same for the full-adder with the following truth table:

a	b	c	s_1	s_0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

(top & bottom K-Maps)

r_i \ $a_i b_i$	00	01	11	10
0	0	1	0	1
1	1	0	1	0

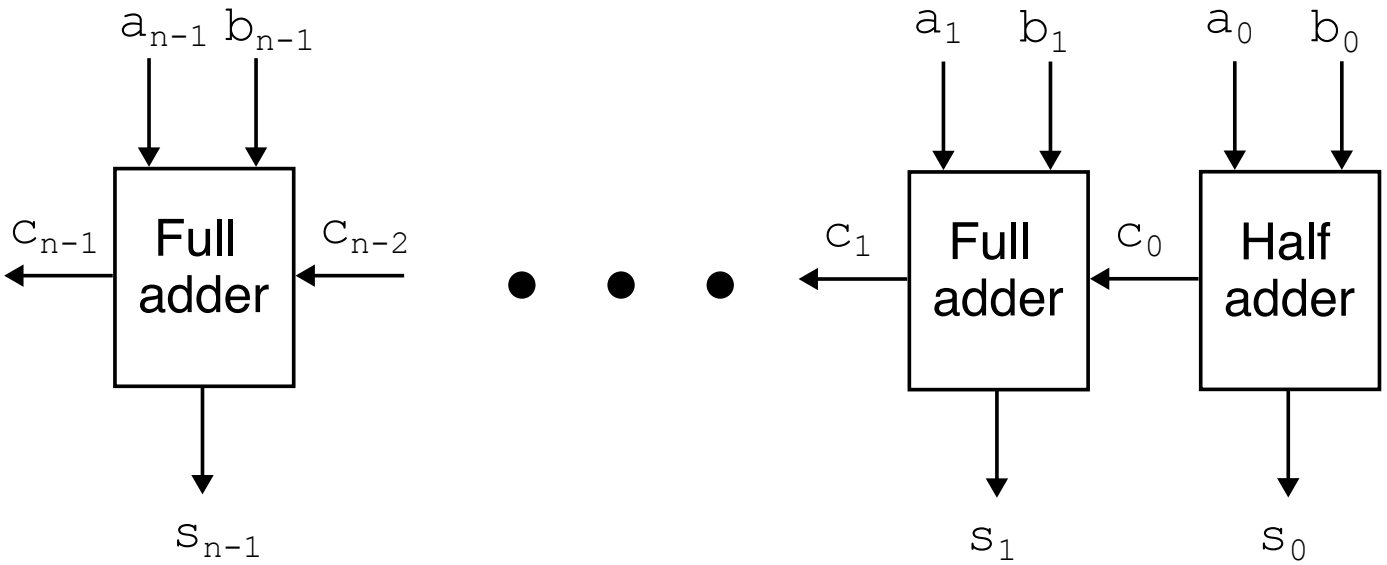
$$s_0 = a_i \oplus b_i \oplus r_i$$

r_i \ $a_i b_i$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$s_1 = a_i b_i + r_i b_i + r_i a_i$$

Addition circuit for 2 words of n bits

- LSB sum can be computed using one half-adder circuit using LSBs of a, b as operands
- Other $n-1$ bits can be computed using $n-1$ full adder circuits $\rightarrow$ n circuits are assembled in series for a complete n-bit adder
- This circuit is known as ripple carry adder, carry needs to propagate from LSB to MSB



We know why the above circuit will not be good for $n >>$!

Can we do better?

- Of course ... addition is a very common problem, and there are many different ways how this could be done, with different parameter trade-off, but out of scope here
- However let's consider another implementation strategy: imagine the whole addition problem as one combinatorial problem, input words A and B are considered as input to our system, i.e., word bits are contacted as seen as problem input
- Example of an added with 2-bits per word $\rightarrow$ combinatorial problem with truth table of 16 lines (this is easy ...)
- For a sum $(3)_{10} + (3)_{10} = (6)_{10}$, we need 3-bits of the result, so 3 bits of output i.e., 3 logic functions (see next slide for the truth table)
- Arithmetic sum of two n bit words is producing $n+1$ bits of result

What is the major problem of this approach?

Truth table for 2× 2-bit words (incomplete)

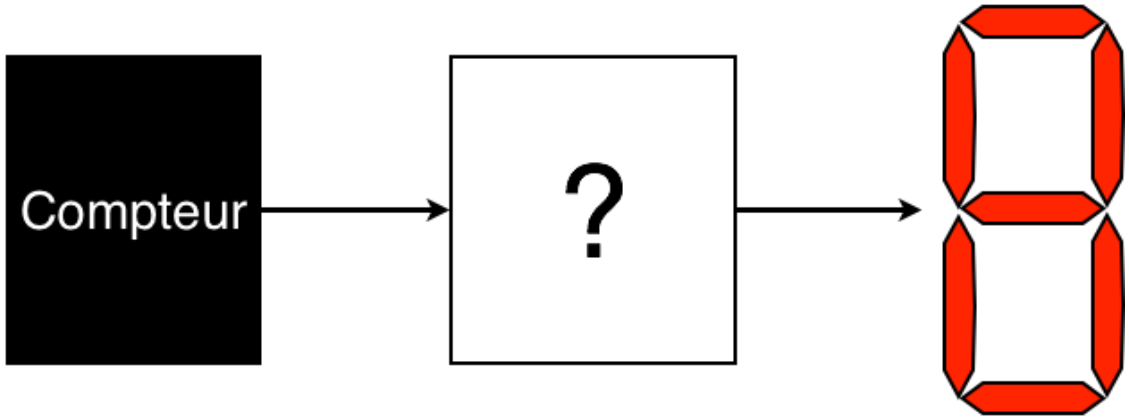
a_1	a_0	b_1	b_0	s_1	s_1	s_0
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	1	1
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	1	1
0	1	1	1	1	0	0
	...				...	
	...				...	
	...				...	

Could you complete the table and derive functions?

3. Circuits with multiple outputs

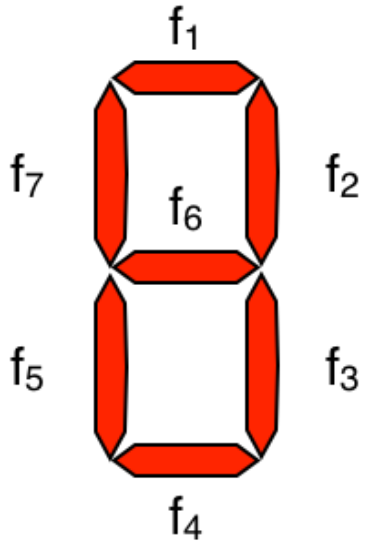
Example 1: 7 segments display

- Imagine a 4-bits word to be printed using a 7 segment display, composed of 7 independent LEDs
  - Input 4-bits word could be generated by anything (keyboard, ADC etc.); here let's assume that the output of a simple BCD counter drives the display
- Each LED can be turned `ON` or `OFF` using a Boolean function f_i; say if $f_i = 1$ then the LED is on, otherwise it is off
- Our task is to design a circuit (establish optimised logic functions) that will convert the input to the appropriate LED configuration



Combinatorial circuit synthesis

- The circuit is clearly combinatorial, same input combination should produce the same output
- There are 4 single bit inputs & 7 different logic functions
- We assign a logic function f_i to each LED: this is arbitrary, but needs to be respected from the beginning to the end of the synthesis & physical implementation process



- The circuit is clearly combinatorial, same input combination **should always produce the same output** – combinatorial circuit
- How to construct truth tables?

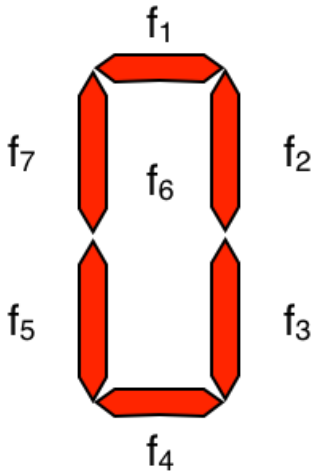
How to proceed?

- Imagine that we want to print digit 0 $\rightarrow$
  - Following segments need to be turned on:

$$f_1 = f_2 = f_3 = f_4 = f_5 = f_7 = 1$$

  - And the following segments need to be off:

$$f_6 = 0$$

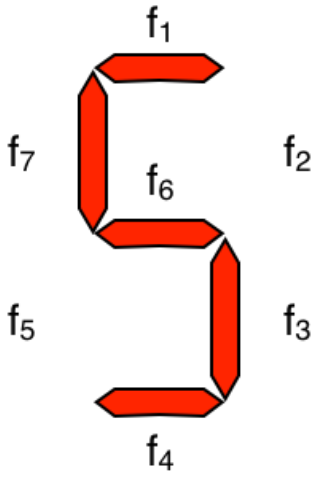


- Similarly if want to print digit 5 $\rightarrow$
  - Following segments need to be turned on:

$$f_1 = f_3 = f_4 = f_6 = f_7 = 1$$

  - And following segments need to be off:

$$f_2 = f_5 = 0$$



- We can continue for **all input combinations**; combinatorial problems should be explored exhaustively

Truth tables for 7 outputs

- Table below covers possible input combinations from 0-9

Decimal	f_1	f_2	f_3	f_4	f_5	f_6	f_7
0	1	1	1	1	1	0	1
1	0	1	1	0	0	0	0
2	1	1	0	1	1	1	0
3	1	1	1	1	0	1	0
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1
7	1	1	1	0	0	0	0
8	1	1	1	1	1	1	1
9	1	1	1	1	0	1	1
10							
11							
12							
13							
14							
15							

- The problem has 4 inputs and the truth table has 2^4 lines; codes $(10)_{10} \rightarrow (15)_{10}$ are not used !!!
- What do we do with these input combinations?

Solution 1: use "don't cares"

- Set the outputs to "don't cares" for all inputs combinations that have not been covered by the specification
- We write in all unfilled truth table cells the 3rd symbol of the B, famous $-$; Do you remember final set B? ($B \in \{0, 1, -\}$)
- With incompletely specified functions & "don't cares" we can use '0' or '1' for the output for a given input combination, whatever gives us the best possible simplification during logic synthesis

- If we do so, and for that particular input situation, the circuit will turn on certain segments depending what we have used for our simplification ... and we could end up with something meaningless
- If we can (or can't live) with such solution will depend on what we really want to do

Solution 2: use "don't happen"

- If instead of "don't care" we speak of "don't happen", then we will never have a problem of the erroneous print
- Numbers 10-15 will simply never appear as input combination to our circuit, and we will always have the correct output
- How to practically implement "don't happen"?
  - Example: if it is a 4-bit counter that feeds our circuit we set the current value of the counter to 0 explicitly using some kind of `RESET` signal once we reach 10
  - Sometimes we don't have to do anything to implement "don't happen", they are present by the nature of the problem or system that we look into
  - Above situation occur when some sensors **can't be true at the same time**; imagine an elevator and the sensors that signal the presence at the 1st and the last floor; input combination where both of the above are true is not possible (or if this is the case, then there is a problem with sensors themselves)

Solution 3: set a specific value

- We print an "error" message if input is > 9 using a symbol E; same output set for input combinations 10-15

Decimal	f_1	f_2	f_3	f_4	f_5	f_6	f_7
	...	...	...	...	...	...	
10	1	0	0	1	1	1	1
11	1	0	0	1	1	1	1
12	1	0	0	1	1	1	1
13	1	0	0	1	1	1	1
14	1	0	0	1	1	1	1
15	1	0	0	1	1	1	1

If you adopt this solution, what will be the consequence on the synthesised functions compared to those using "don't care" or "don't happen" solution?

Example 2: BCD $\rightarrow$ Excess-3 converter

- Specification – design a logic circuit that converts 4-bit input word representing a decimal BCD number into Excess-3 code; 4-bit input word is generated by another circuit (e.g. counter)
  - Establish truth table of the problem
  - Provide a list of mintermes & maxtermes (their decimal equivalents)
  - Draw K-Maps for all outputs
  - Provide the list of all Prime Implicants
  - Simplify the logic expression (draw the corresponding circuits)
- Problem analysis
  - Inputs: *...converts a 4-bits input word...*: there are 4 inputs, truth table(s) will have $2^4 = 16$ lines
  - Outputs: *...code Excess-3...*: BCD digits 0 to 9 are encoded using 4 bits; we have 4 bits of output
- There are 4 logic functions, each with 4 variables

Conversion table BCD → Excess-3

- Excess-3 is a simple code, 4th digit is 2 and not 8; following Look Up Table provides the conversion key:

Decimal	8421 (BCD)	2421
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	1011
6	0110	1100
7	0111	1101
8	1000	1110
9	1001	1111

- Excess-3 is self-complementary code (there is a symmetry)
- Table is not complete, remaining 6 input combinations (10-15) set to "don't cares" (or "don't happen" if input is never >9)

Complete truth tables: BCD $\rightarrow$ Excess-3

- There are 4 truth tables here, each with 4 variables

Decimal	BCD	2421
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	1011
6	0110	1100
7	0111	1101
8	1000	1110
9	1001	1111
10	1010	- - -
11	1011	- - -
12	1100	- - -
13	1101	- - -
14	1110	- - -
15	1111	- - -

K-maps 1/2

x_4x_3 \ x_2x_1	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	-	-	-	-
10	0	1	-	-

x_4x_3 \ x_2x_1	00	01	11	10
00	0	0	1	1
01	0	1	0	0
11	-	-	-	-
10	1	1	-	-

$$f_1 = x_1$$

$$f_2 = x_4 + x_3x_2'x_1 + x_3'x_2$$

All Prime Implicants are essential!

Can you tell why?

K-maps 2/2

x_4x_3 \ x_2x_1	00	01	11	10
00	0	0	0	0
01	1	0	1	1
11	-	-	-	-
10	1	1	-	-

x_4x_3 \ x_2x_1	00	01	11	10
00	0	0	0	0
01	0	1	1	1
11	-	-	-	-
10	1	1	-	-

$$f_3 = x_4 + x_3x_2 + x_3x_1'$$

$$f_4 = x_4 + x_3x_1 + x_3x_2$$

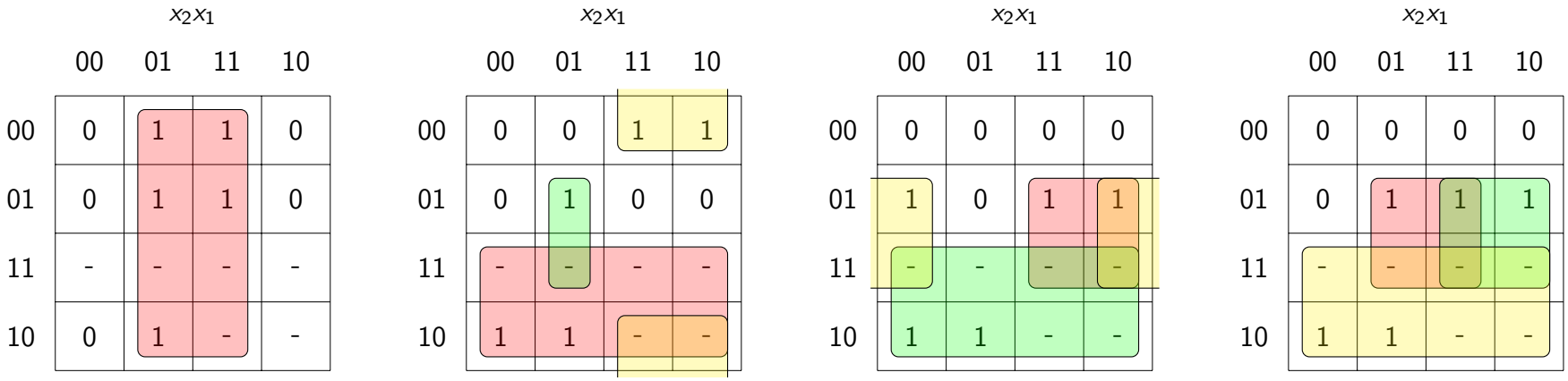
All Prime Implicants are essential!

Can you tell why?

Decimal equivalents

- More compact notation with decimal equivalents

$$
\begin{aligned}
f_1 &= \sum m(1,3,5,7,9) + \sum d(10,11,12,13,14,15) \\
f_2 &= \sum m(2,3,5,8,9) + \sum d(10,11,12,13,14,15) \\
f_3 &= \sum m(4,6,7,8,9) + \sum d(10,11,12,13,14,15) \\
f_4 &= \sum m(5,6,7,8,9) + \sum d(10,11,12,13,14,15)
\end{aligned}
$$



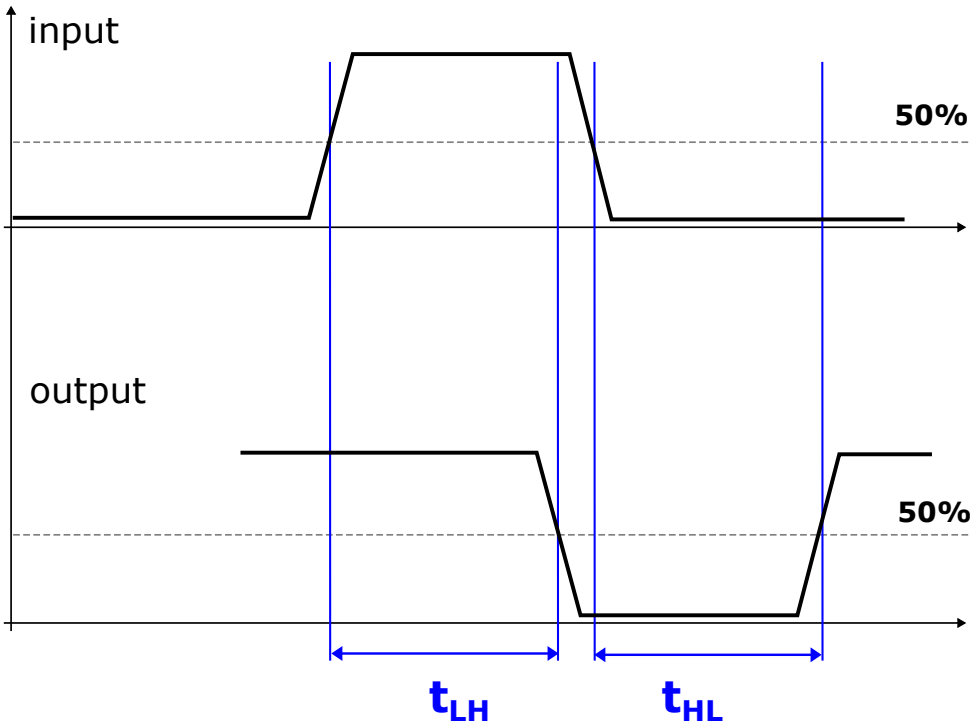
4. Hazards

Hazards occur in real-world circuits

- Until now we have considered our logic circuits to be ideal: zero input, gate and/or wire delays; transistors switch instantly, signals propagate through wires with no delay, there is no rising time
- Above hypothesis has an important consequence: when input changes (e.g. Alice changes inputs), the output is produced instantaneously, i.e. Sum-of-Products circuit (i.e. AND and OR gates) will respond immediately
- Obviously in the real world this is not the case; no matter how fast is your circuit, there will be always delays
- If we want the observer (human or another circuit) to see the correct output, these delays must be accounted for; if not, observer will see previous, non-updated value
- On top, delays are not the same across the circuit, they will vary from one component to another, & in different parts of the circuit

Individual gate switching time

- Delays are due to R, L, C of devices used to build gates & wires
- Gate switching time is defined as the time required for an input to provoke the change at the output (`INV` gate example below)

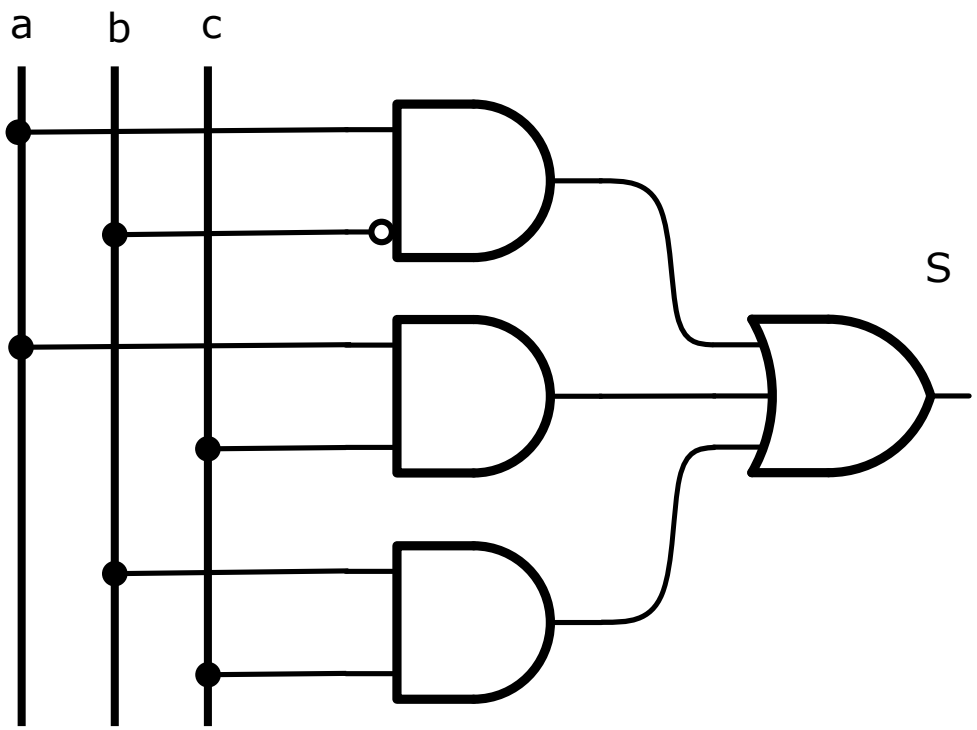


$$t = \frac{t_{LH} + t_{HL}}{2}$$

- Wires also have certain R, L, C, that are function of the wire length; wire delays will have to be added to the delay of gates

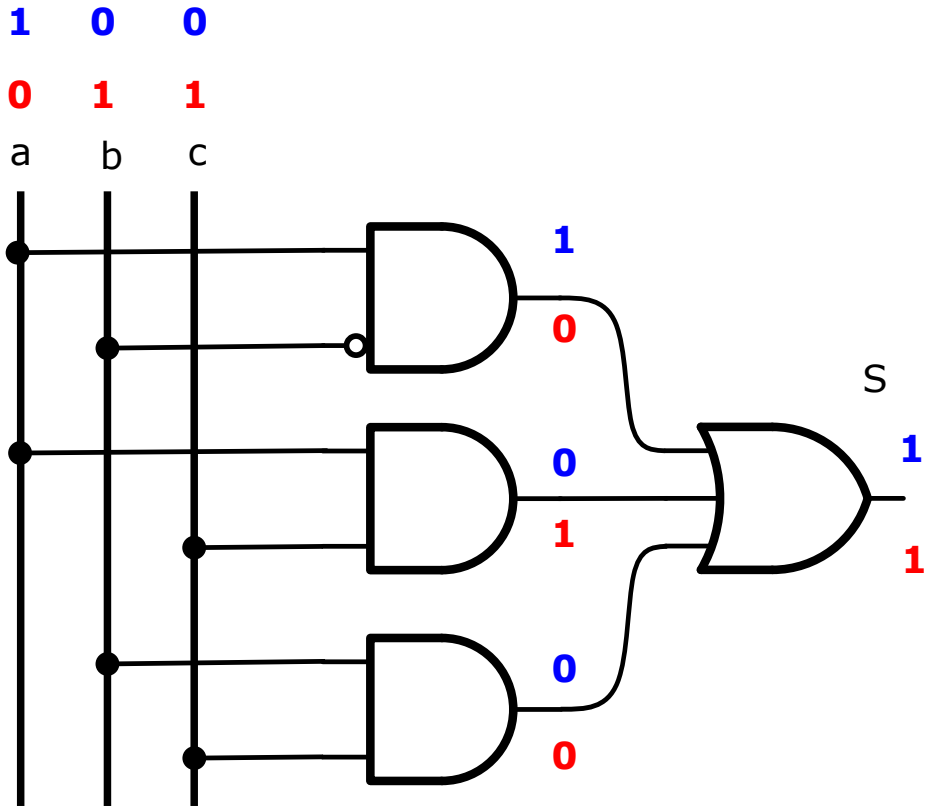
Gate delays and end result computation

- Problem: due to circuit differences & variations, different logic paths are not going to have the same delay
- Looking at the circuit below certain `AND` gates could switch earlier, and other later
- Other than wrong output this will cause other problems ...



Two gates switching simultaneously

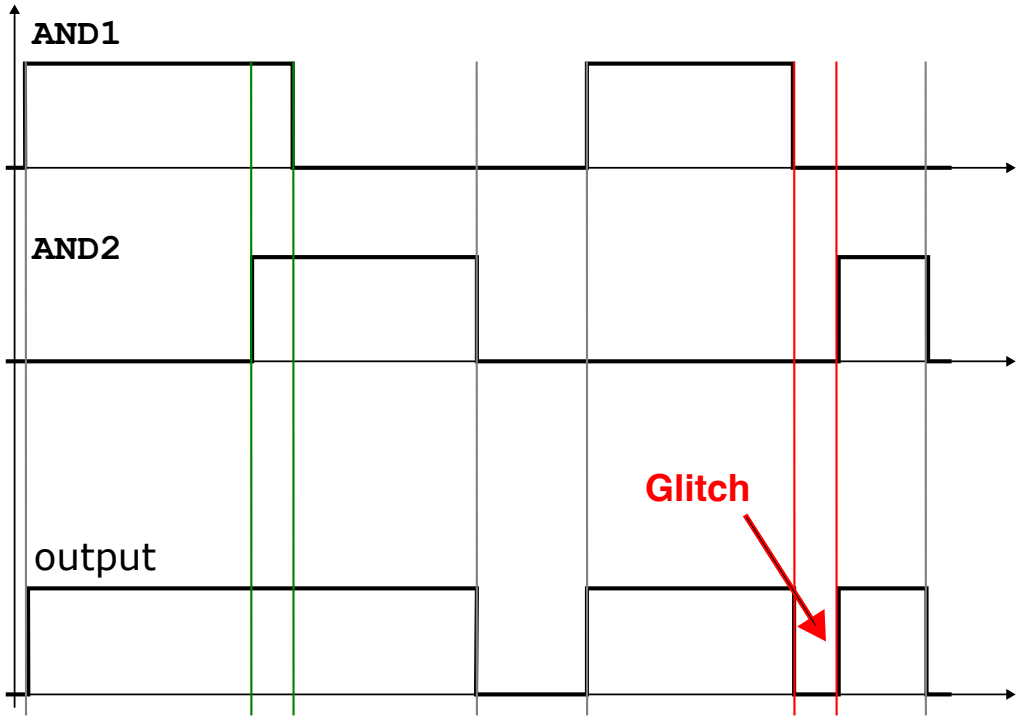
- Let us imagine the previous circuit with the following input: $abc = 100$ that causes the first AND to output logic '1'
- Imagine now that Alice introduces new input $abc = 011$



- First gate switches from 1 to 0, & the second gate from 0 to 1; in perfect world these two are simultaneous, output remains '1'

Two gates switching non-simultaneously

- But gates have different switching delays, 2 situations may occur:
  - AND2 turns on before gate AND1 turns off → No problem!
  - AND2 turns on after AND1 turned off ... → Problem



Assuming that OR reacts quickly enough (this is possible), during a short amount of time we will see a zero on output – glitch

Hazards

- If the situation described in the previous slide occurs, we speak of glitches or hazards defined as non-desired impulses of a (very) short duration
- Important question to ask: are glitches safe or not?
- It depends on what we do with the produced output
- If the output is used by someone or something not sensible to such variations – we do not care!
  - Human eyes sample reality with a period of 1/30s; if the output drives an LED, and 2ps glitch occurs, there is no way it will be perceived in any way
  - But digital systems are used to drive another digital systems, and their sampling rate may be way better; if controlled system is another digital system glitch can enter the digital logic and it can be captured by the circuitry to produce an unwanted output, including something bad
- In general we do everything to avoid glitches (hazards) in circuits

Solution to hazards

- Hazards can be avoided by adding redundant AND gates that maintains the output to '1' during transition of other AND gates
- In K-Maps extra term should cover the interface between the red & the green n-cubes below (each n-cube results in an AND gate):

ab \ cd	00	01	11	10
00	1	1	1	0
01	1	1	1	0
11	0	0	1	0
10	0	0	1	0

- We add extra term $a'd$ to

$$F = a'c' + cd + a'd$$

- It may look strange that we add extra gates after optimization effort, but this is a mandatory evil to enable workable circuits

Remarks

- By default you should not add any redundant terms, especially when you do logic optimisation; add redundant terms only when you are asked to do so!
- Attention! Proposed method will not solve all possible sources of glitches; other methods will be deployed (out of scope here)
- Avoiding glitches is a more general problem in logic (integrated) circuit design → Signal Integrity (SI)
  - Imagine multiple wires running in parallel on a certain distance and physically being very close
  - These wires could carry high frequency switching signals (in logic circuits these days this will happen for sure ...)
  - If one wire switches (aggressor) with high frequency and the other not (victim) we may have coupling and glitches may be induced into the victim wire

Solving SI issues is a very important activity when designing integrated circuits today

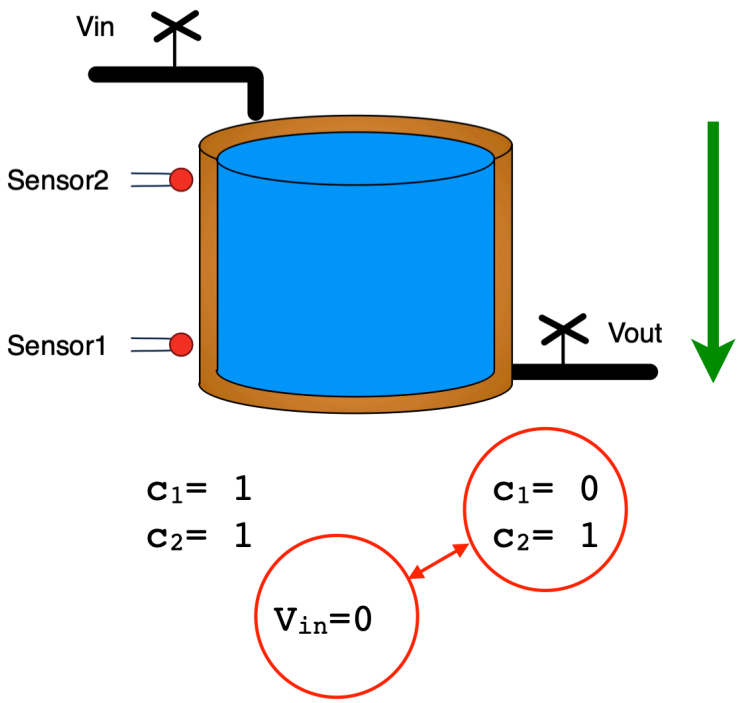
5. Introduction to sequential systems

These lectures: ToC revisited – 3 parts

- Boolean algebra
  - Arithmetic tools used to write & manipulate Boolean logic equations
  - Various methods of logic optimisation to enable efficient physical system design (physical refers to the assembly of actual gates)
  - Link between Boolean logic equations & logic circuits
- Combinatorial systems synthesis
  - Representation & manual circuit synthesis from verbal specs
- Sequential Systems
  - Combinatorial systems can't control some very simple processes
  - We need to extend our models of computation & digital systems
  - Introduction of systems with memory & the notion of system state

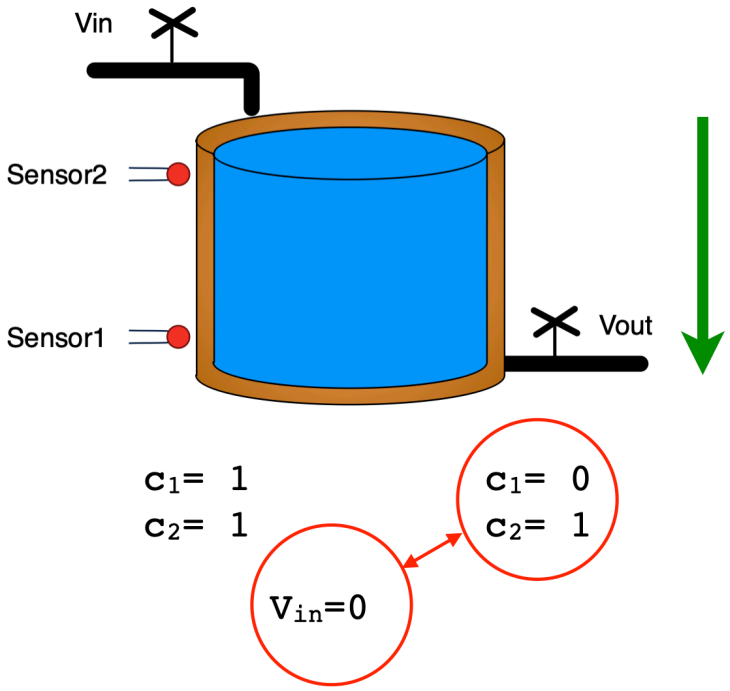
Limitation(s) of combinatorial logic – real system

- Consider the problem of a digital system used to control the process of tank filling with some kind of liquid; to enable the control we have access to two sensors: one sensor on the bottom of the tank, signalling that the tank is empty, and the other one on the top signalling that the tank is full (we are on the earth, so gravity is present)
- System should control the level of the liquid in the tank, so that it never goes below the `Sensor1` or above `Sensor2`, by opening and/or closing the valve V_{in}
- Note that valve V_{out} acts as a random variable emptying the tank at the pace we can't predict in advance (Alice or Bob)



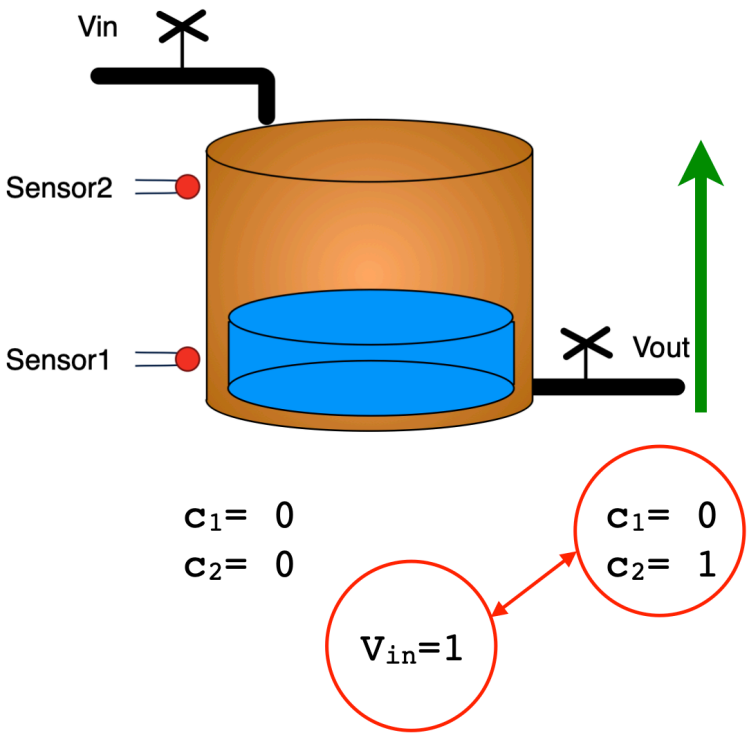
Tank operation

Imagine the tank already filled with liquid, V_{out} is open, V_{in} is closed:



We don't need to open V_{in}

When empty, V_{in} should open & after some time liquid will reach `Sensor1`:



We need to maintain V_{in}!

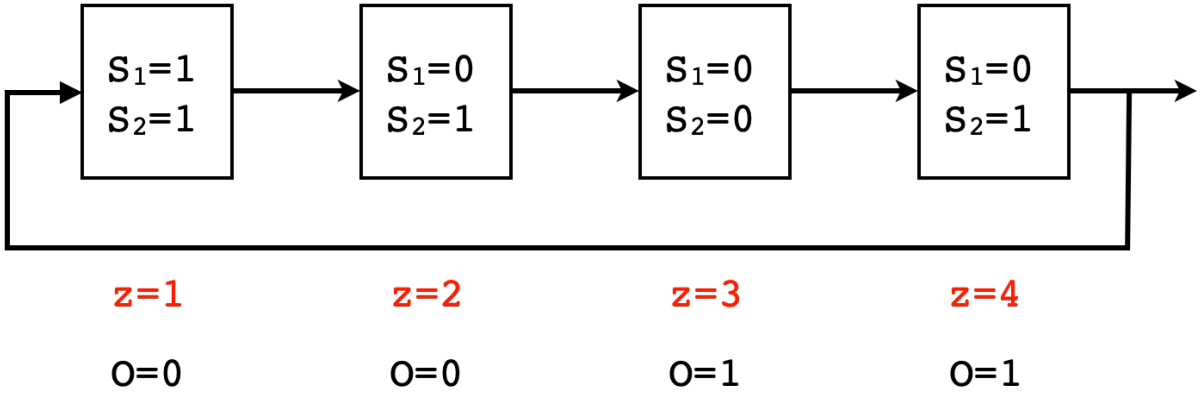
Two different situations for the same combination of inputs!

Limitation(s) of combinatorial systems

- We need to somehow **make the difference** between the following two situations:
  - Tanks is full and is being emptied (V_{in} should remain closed) &
  - Tank is empty and is being filled (V_{in} is opened & should be maintained opened until reaching `Sensor2`)
- System will definitively move from one situation to another, depending on the output valve and sensors, but this can't be decided using inputs only
- In order to be able to distinguish between these two situations, we need to be able to somehow memorise in which of the two situations we are
- We speak of two things here:
  - *"memorise"* – we need some kind of **memory device** to store the situation in which the system currently is
  - *"situation"* – better known as the system state

System evolution as sequence of states

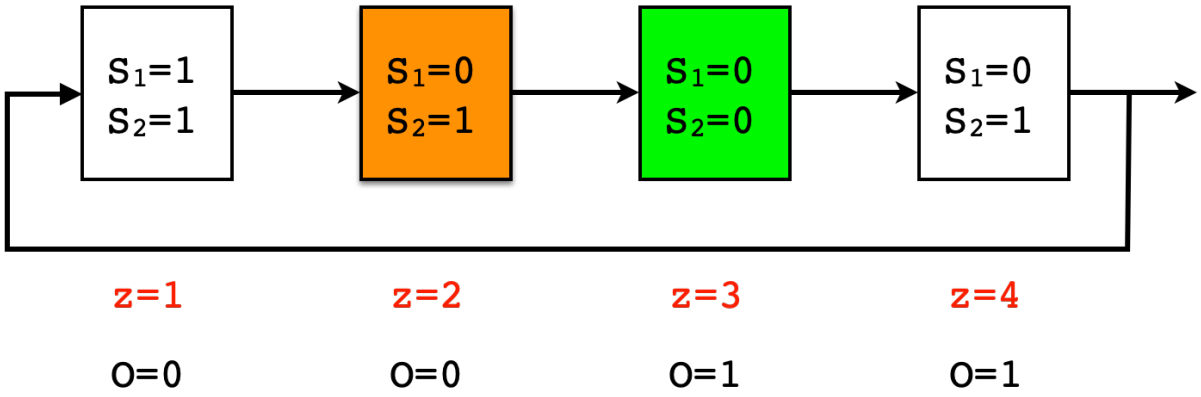
- Let's describe the system using a sequence of states noted with an ID associated to z; the output O will control the V_{in}



  - Tank is initially full (inputs `00`) & it can be emptied $\rightarrow z = 1$
  - If system in $z = 1$ & inputs `01`, system moves to $z = 2$; V_{in} is closed since tank is being full & being emptied (O is maintained at '0')
  - If system in $z = 2$ & inputs `00`, system moves to $z = 3$, i.e. level is the bottom sensor, valve V_{in} may open to fill the tank ($O = 1$)
  - If system in $z = 3$ & inputs `01`, system moves to $z = 4$ we have passed the $Sensor1$ but V_{in} is maintained open with $O = 1$
  - If system in $z = 4$ & inputs `00` we reached initial situation

System evolution: from current to next state

- Imagine system in $z = 2$; this is called a present or current state (orange box); the system will remain in this state as long as the inputs remain the same; that is why we call it stable



- From state $z = 2$, the system **can only go to state** $z = 3$ (and nowhere else!) if the inputs are set to 00
- If $z = 2$ & inputs are 00 the system should go to future state $z = 3$ (green box); there will be state transition: from present state, new input situation causes transition, system moves into a future state
  - Obviously once the transition is completed, predicted future state becomes present state; system moves from one stable state to another depending on the present state & the current inputs

Sequential systems: more formal definition

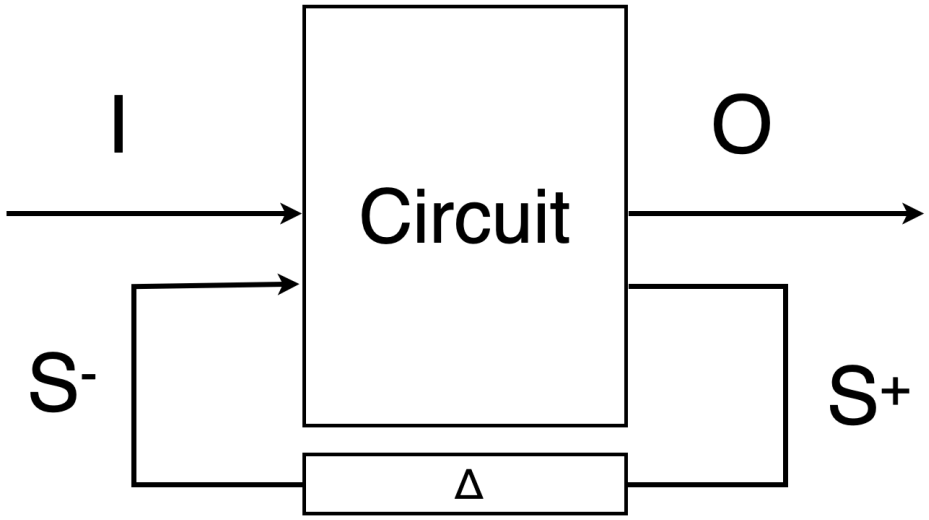
- In sequential systems the **Outputs** (O) depend on **inputs** (I) and system state (S) (so, not only inputs like in combinatorial circuits)
- For the same combination of inputs we can have different outputs
- Future state (S^+) is calculated as a logic function of inputs AND the previous state (S^-) represented using internal variables
- State can be a vector, so multiple Boolean values/expressions
- Sequential logic system is described with two sets of Boolean equations, one set for output(s) and one for next state(s) S^+:

$$\begin{aligned} O &= f_1(I, S^-) \\ S^+ &= f_2(I, S^-) \end{aligned}$$

- Sequential logic circuit can be seen as a combinatorial system with one (or more) feedback(s) on state variables
- Thus all sequential systems have this feedback! (crucial)

Difference: present & future states

- Sequential circuits have feedback loop for the system state (this is how future state becomes present state)
- The difference between present & future states can be modelled with delays between the source and destination points for the state (along the feedback)
- This delay occur in real circuits because of R, L, C in actual wires and gates (within the circuit box)



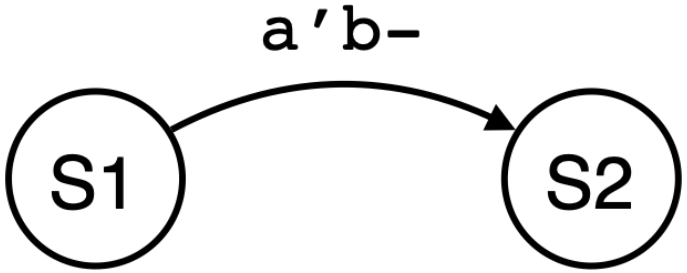
Delay Δ on the wire

Representing sequential systems

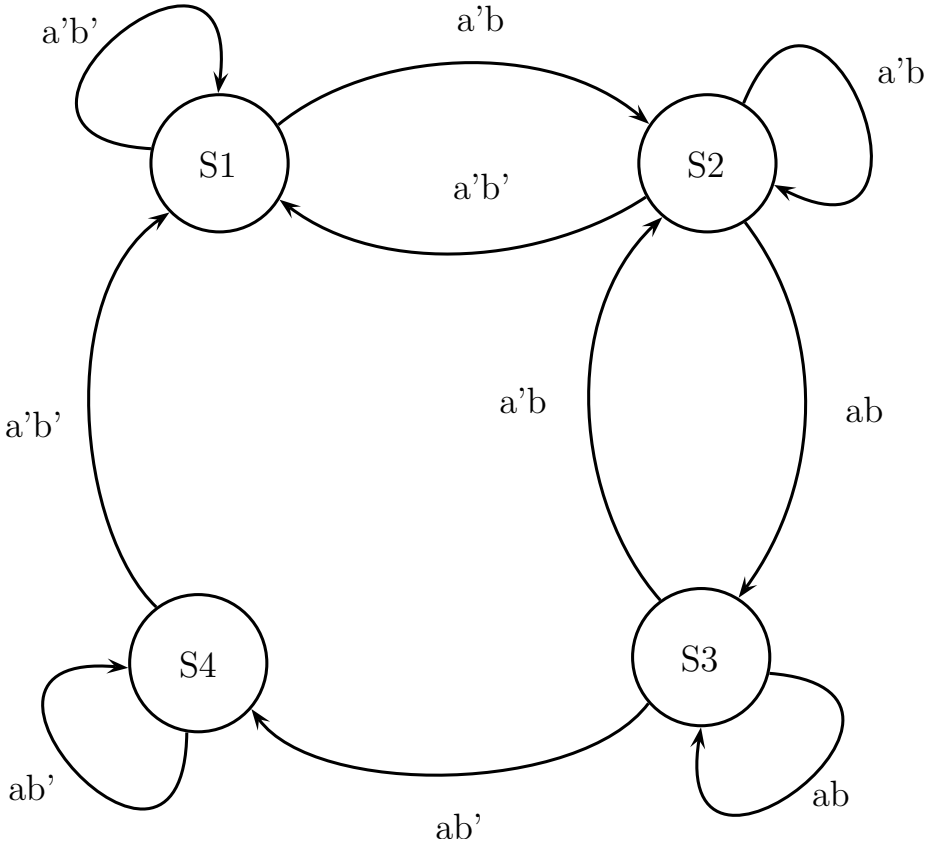
- State graphs – Graphical representation; provides a neat view of all states and state transitions depending on the machine inputs; nice to see, as long as simple; not the preferred mode of representation, especially if we speak of synthesis; it lacks systematic analysis of all possible outcomes
- State table, Hufmann table, transition matrix – tabular representation of all system states and transitions; preferred way of describing state machines (sequential systems) because it forces us to be systematic when synthesising the system (it forces us to to look for all possible outcomes from a given state)
- Logic Equations – Boolean expressions for different logic functions; this is typically derived from the above to build the circuit (these expression will be optimised)

State graphs

- System state is represented with a circle, state name is written inside (state names could be anything ...)
- State transition occurs between two stable states; indicated using oriented arcs (arrows), from present state to the next state, the arrow indicating the transition
- Note that transition condition(s) (on inputs) are written on the top of the arc; we use normal Boolean formalism to do that
- Example – two states S_1 and S_2 with one transition; there are 3 variables a, b, c, since condition is $a'b-$; this means transition will occur when $a = 0$ AND $b = 1$ & whatever the value of c



State graph example: 4 states, 2 inputs



State tables

- Indicates the way the system evolves depending on the present state and inputs; number of lines in the table fixes the number of states (not known in advance)
- First column indicates the present states; other columns indicate all possibilities of input variables, so 2^n for n variables; last column indicates output(s)

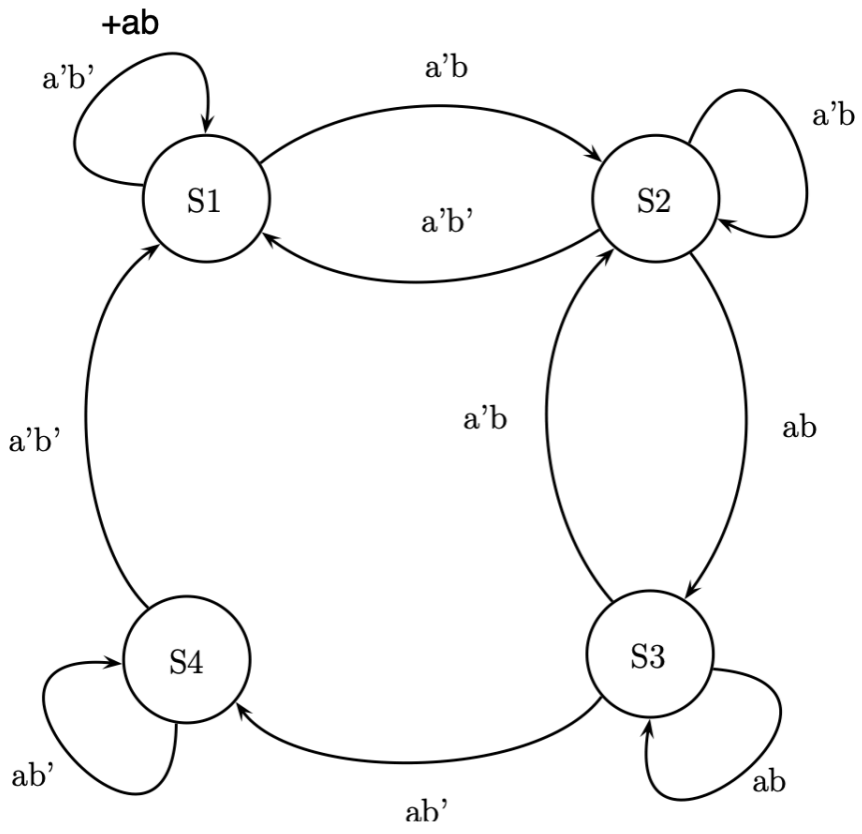
- Stable states are marked in bold (like the circles in state diagrams) system remains in stable state as long as the input remain unchanged
- Transition states are used to indicate possible path from one to another stable state (arcs in the state graphs); transition take place between Source & Destination states

	00	01	11	10	Z
1	**1**	2			1
2		**2**			0
3					
4					
5					

ab

State graph to state table

- We can easily switch from one representation to another:



	00	01	11	10
1	**1**	2	**1**	–
2	1	**2**	3	–
3	–	2	**3**	4
4	1	–	–	4

Note that non-existing transitions are marked using don't cares (e.g. from S1 to S3)

How to read the state table

- System is in stable state `1` (marked in bold), inputs *ab* are set to `00`, system remains in this state `1` as long as inputs remain `00`
- Imagine now that *ab* changes from `00` to `01`, Alice (or worse, Donald!) pushed the button
- System "leaves" the current stable state; we switch the column, this new situation corresponds to new input combination (we remain in the same line of the table though, because of the present state)
- We read what is written in this new cell & this becomes our new destination
- New destination could be a stable state (system remains in that state) or a transition (normal text); if it is a transition, system continues until it reaches a stable state (here state `4`)

ab

	00	01	11	10
1	**1**	2	2	
2		3	**2**	
3		4	**3**	
4		**4**	3	

Important hypothesis for Alice

- In a state table the number of transitions between two stable states is not limited; however it is mandatory that inputs don't change during transitions (transitions are not instantaneous!)
- If this hypothesis is not satisfied the system behaviour is not guaranteed any more; we want to avoid this at any price

- Imagine a transition from state `1` to `4` passing through transitions `2`, `3` & `4`, & triggered by input sequence `00`, `01`, `11`
- Imagine that Alice pushes the button `a` during transition `3`, the system will end in state `2`, which is stable (system will remain there until the next input change)
- For input sequence `00`, `01`, `11` we ended up in state `2` and not `3`

	ab				
	00	01	11	10	Z
1	**1**	2			
2		3	**2**		
3		4	**3**		
4		**4**	3		